

DDH-based schemes for multi-party Function Secret Sharing

Marc Damie^{1,2}^{*}, Florian Hahn¹, Andreas Peter³, and Jan Ramon²

¹ University of Twente, The Netherlands

² Inria, France

³ Carl von Ossietzky Universität Oldenburg, Germany

Abstract. Function Secret Sharing (FSS) schemes enable sharing efficiently secret functions. Schemes dedicated to point functions, referred to as Distributed Point Functions (DPFs), are the center of FSS literature thanks to their numerous applications including private information retrieval, anonymous communications, and machine learning. While two-party DPFs benefit from schemes with logarithmic key sizes, multi-party DPFs have seen limited advancements: $O(\sqrt{N})$ key sizes (with N , the function domain size) and/or exponential factors in the key size.

We propose a DDH-based technique reducing the key size of existing multi-party schemes. In particular, we build an honest-majority DPF with $O(\sqrt[3]{N})$ key size. Our benchmark highlights key sizes up to $10\times$ smaller (on realistic problem sizes) than state-of-the-art schemes. Finally, we extend our technique to schemes supporting comparison functions.

Keywords: Function Secret Sharing · FSS · Distributed Point Function · DPF · DDH · Multi-Party Computations.

1 Introduction

Secret sharing [28] is a popular cryptographic primitive to perform multi-party computations (MPC) [16]. This primitive enables splitting a secret value into several shares, revealing individually no information about the secret. While classic secret sharing focused on scalar secret values, Function Secret Sharing (FSS) [17] generalizes the concept to share secret functions. FSS schemes consists in three algorithms: Gen, Eval, and Decode. Gen outputs p FSS keys based on a secret function f ; each shareholder receives one key. Eval takes as input an FSS key k_i and a point x , and outputs a share of $f(x)$ (referred to as $\llbracket f(x) \rrbracket_i$). Finally, Decode takes as input p shares $\{\llbracket f(x) \rrbracket_1, \dots, \llbracket f(x) \rrbracket_p\}$ and outputs $f(x)$.

Significant efforts [4,5,10,12,17] aimed to share point functions (i.e., a function equal to zero everywhere, except on a point α). These so-called “Distributed Point Functions” (DPFs) were initially proposed to build private information retrieval (PIR) protocols [17] and have been later used in other domains such as anonymous communications [12] or privacy-preserving machine learning [29].

^{*} Corresponding author: m.f.d.damie@utwente.nl

Recently, there is also a growing interest in schemes supporting comparison functions: functions such that $f(x) = \beta$ when $x \leq \alpha$, 0 otherwise. These schemes called “Distributed Comparison Functions” (DCF) have applications notably in MPC pre-computations [6] or private statistics [1].

Related works The main goal of FSS works is to minimize the key size. In particular, existing works systematically studied the influence of the function domain size N on the key size. While schemes are often described for a generic group $\mathbb{G}$, many works focused on bit-string outputs [4,10]: $\mathbb{G} = (\mathbb{F}_2)^l$; motivated by applications in PIR. However, novel applications in private statistics [1,2] or MPC precomputations [6] require prime fields $\mathbb{F}_q$ (e.g., to sum several shared functions). Such constraints emphasized under-studied scalability issues in some multi-party schemes [4,10] that have an exponential dependency in q . Recent works [3,22] also highlighted this issue.

2/3-party DPF Boyle et al. [4] presented the reference 2-party scheme, with an $O(\log N \log q)$ key size. This scheme has been further optimized by various works: key size optimization [5], “incremental” DPF [2], reusable keys [8], MPC-based key generation [15], verifiable DPF [11]. Thanks to its logarithmic key size, this scheme is used in many application papers including in anonymous communications [25], private statistics [2], and access control [27].

Bunn et al. [9] described a three-party scheme with $O(\sqrt{N})$ key size. Later, [10,30] further improved it to obtain $O(\log N)$ key sizes.

Multi-party DPF Now that 2-party and 3-party schemes have logarithmic key sizes, improving arbitrary p -party schemes is the natural next step. As we focus on multi-party FSS, we will skim through existing schemes, their advantages and disadvantages. Table 1 of Section 3 summarizes this overview.

First, Boyle et al. [4] designed a scheme based on Pseudo-Random Generators (PRG) with $O(\sqrt{N} q^{\frac{p-1}{2}} \log q)$ key size. The exponential factor $q^{\frac{p-1}{2}}$ (with $q = |\mathbb{G}|$) makes it impractical for applications requiring arbitrary output groups (e.g., private histograms [2]). This weakness was already highlighted by [22].

Second, Corrigan-Gibbs et al. [12] introduced a DDH-based scheme with $O(\sqrt{N} \log q)$ key size. Recently, Kumar et al. [22] further improved this scheme by removing a constant factor, but requires a *trusted share decoder*.

Third, Bunn et al. [10] designed an honest-majority scheme (based on [4]) with $O(\sqrt[4]{N} q^{\frac{p-1}{2}} \log q)$ key size. It has the best dependency on N (i.e., $O(\sqrt[4]{N})$), but it inherits the exponential factor of [4]. Papers presenting PRG-based schemes [4,10] only provide algorithms for $q = 2$, but they can easily be generalized to an arbitrary q (as explained in [3]).

Fourth, Bunn et al. [10] also introduced an honest-majority scheme with $O(\sqrt{N} \log q)$ key size. The main advantage is **its information-theoretic security**. Other works also investigated information-theoretic DPF schemes [7,21,23], but [10] is the only with practical key sizes.

Finally, concurrently to our work, two other papers [18,20] have improved multiparty DPF. On the one hand, Goel et al. [18] built a PRG-based scheme

replacing the exponential factor present in [4] with a polynomial factor: $O(\sqrt{N} \cdot p^3 \lambda^4)$. On the other hand, Krips and Pullonen-Raudvere [20] introduced two new hardness assumptions to build a multi-party scheme with logarithmic key size.

DCF Boyle et al. [4] adapted their 2-party DPF to build a DCF scheme with $O(\log N \log q)$ key size. They also adapted their multi-party DPF to build a multi-party DCF with $O(\sqrt{N} q^{\frac{p-1}{2}} \log q)$; suffering from the same scalability issues as their DPF. Recently, Kumar et al. [22] proposed multi-party DCF inspired by the DPF of [12]. The literature describes no other DCF; the DCF baselines are then weaker than those in DPF.

Gap in the literature When relying on standard hardness assumptions (which then excludes [20]), the existing literature leaves two choices for multi-party DPF: either an $O(\sqrt[3]{N})$ key size with exponential factors [10], or an $O(\sqrt{N})$ key size without exponential factors [10,12,14].

Our goal is to provide an intermediary solution, a better trade-off: $O(\sqrt[3]{N})$ key size without exponential factors. Similarly, we want to extend these results to DCF, and provide the same choice range as in DPF.

We would like to emphasize the contribution of [20] which, for the first time, presents a multi-party scheme with logarithmic key size. Unfortunately, it requires the introduction of two new hardness assumptions. Further research is necessary to estimate the parameters under which these assumptions hold. Our paper, on the other hand, relies on standard, widely-established assumptions.

Our Contributions

1. We build a **multi-party DDH-based DPF with $O(\sqrt[3]{N})$ key size**. Our scheme provides keys **up to $\times 10$ smaller** than existing works.
2. We **extend our approach to build a DCF** with $O(\sqrt[3]{N})$ key size.
3. As DDH-based schemes require encoding the secret as a DDH group element, we present **two encodings, discuss their properties and applications**.

2 Definitions

Let p be the number of parties/shareholders, m be the number of dishonest parties. Let $\mathbb{F}_q$ be a prime field and let $\mathbb{G}$ be a cyclic group. Let N be the function domain size and 1^λ a security parameter. Finally, let $s \xleftarrow{R} S$ be the uniform sampling of an element s from the set S . Let $\llbracket x \rrbracket$ be a share of x , and $g^{\llbracket x \rrbracket}$ to g to the power $\llbracket x \rrbracket$.

2.1 Threat model

MPC protocols are referred to as secure if they preserve output correctness and input privacy in presence of an adversary. Subsection 2.2 provides definitions of correctness and privacy specific to FSS.

Like most FSS works [4,5,10,17,22], we focus on semi-honest adversaries; adversaries following the protocol and *passively* inferring secret information.

Moreover, secure protocols are characterized by the tuple (m, p) . The security of an (m, p) -secure protocol is guaranteed only if the number of dishonest parties is below than or equal to m . The literature distinguishes honest-majority protocols ($m < p/2$) from dishonest-majority protocols ($m \geq p/2$).

2.2 Function Secret Sharing

Function secret sharing (FSS) [4] generalizes the concept of secret sharing to secret functions. Each FSS scheme can share functions from a specific function family. A function family $\mathcal{F}$ [3] is a pair $(P_{\mathcal{F}}, E_{\mathcal{F}})$ where $P_{\mathcal{F}} \subseteq \{0, 1\}^*$ is an infinite collection of function descriptions $\hat{f}$, and $E_{\mathcal{F}}: P_{\mathcal{F}} \times \{0, 1\}^* \rightarrow \{0, 1\}^*$ is a polynomial-time algorithm defining the function described by $\hat{f}$.

In other words, each function description $\hat{f} \in P_{\mathcal{F}}$ describes a corresponding function f such that $f(x) = E_{\mathcal{F}}(\hat{f}, x)$. A description $\hat{f}$ for such functions is the tuple $(\alpha, \beta, \mathcal{X}, \mathcal{Y})$, with $\mathcal{X}$ the function domain and $\mathcal{Y}$ the output space.

Over the years, the literature has described schemes for various function families especially point functions [4,5,7,10,12,17] (functions f such that $f(x) = \beta$ if $x = \alpha$, and $f(x) = 0$ otherwise), and comparison functions [4,22]. Other function families such as decision trees [5] have FSS schemes, but they have fewer applications than DPF and DCF.

All functions within a function family must share the same domain and output space. Moreover, we use the notation $\mathcal{K}$ to refer to the FSS key space.

Definition 1. *A p -party FSS scheme (for a family $\mathcal{F}$) has 3 algorithms:*

- $\text{Gen} : \mathbb{N} \times P_{\mathcal{F}} \rightarrow \mathcal{K}^p$ takes as input a security parameter $1^\lambda \in \mathbb{N}$ and a function description $\hat{f} \in P_{\mathcal{F}}$, and outputs p keys $k_1, \dots, k_p \in \mathcal{K}$.
- $\text{Eval} : \mathcal{K} \times \mathcal{X} \rightarrow \mathbb{G}$ takes as input k_i and a point $x \in \mathcal{X}$, outputs a share of $f(x)$ that we denote as $\llbracket f(x) \rrbracket_i$.
- $\text{Decode} : \mathbb{G}^p \rightarrow \mathcal{Y}$ takes as input p shares $\{\llbracket f(x) \rrbracket_1, \dots, \llbracket f(x) \rrbracket_p\}$ and outputs the secret $f(x)$.

Recent FSS works [10,30] generalized this definition to support threshold secret-sharing. Our work (like most existing works [4,5,10,12]) does not require such a generalization, so we stick to the classic definitions from [4].

Definition 2 (Correctness [4]). *For any function $f \in \mathcal{F}$, for any point $x \in \mathcal{X}$, we have $k_1, \dots, k_p \leftarrow \text{Gen}(1^\lambda, \hat{f})$ and*

$$\mathbb{P}[\text{Decode}(\text{Eval}(k_1, x), \dots, \text{Eval}(k_p, x)) = f(x)] = 1$$

Definition 3 (Privacy [3]). *Let $\text{Leak} : \{0, 1\}^* \rightarrow \{0, 1\}^*$ be a function specifying the allowable leakage. If omitted, it is understood to be $\text{Leak}(\hat{f}) = (\mathcal{X}, \mathcal{Y})$.*

We call a p -party FSS scheme private if, for every set of corrupted parties $S \subseteq \{1 \dots p\}$ of size m , there exists a PPT algorithm Sim (simulator), such

that for every sequence of function descriptions from $P_{\mathcal{F}}$ ($\hat{f}_1, \hat{f}_2, \dots$) of size polynomial in λ , the outputs of the following experiments Real and Ideal are computationally indistinguishable:

- Real(1^λ) : $(k_1, \dots, k_p) \leftarrow \text{Gen}(1^\lambda, \hat{f}_\lambda)$; Output $(k_i)_{i \in S}$
- Ideal(1^λ) : Output $\text{Sim}(1^\lambda, \text{Leak}(\hat{f}_\lambda))$

3 DDH-based Distributed Point Function

This section introduces a DDH-based approach to build a scheme with $O(\sqrt[3]{N})$ key size upon the information-theoretic scheme of [10] which has an $O(\sqrt[3]{N})$ key size. We call “sub-DPF” or “sub-scheme” the DPF scheme of [10] on which we apply our DDH-based optimization.

As this section focuses on point functions, we use the notation (α, β) to refer to the parameters of the secret function f (i.e., $f(x) = \beta$ if $x = \alpha$, 0 otherwise).

3.1 Scheme

We represent the domain as a grid of dimensions: $(\sqrt[3]{N})^2 \times \sqrt[3]{N}$ (see Figure 1). The non-zero value is in cell (γ_*, δ_*) .

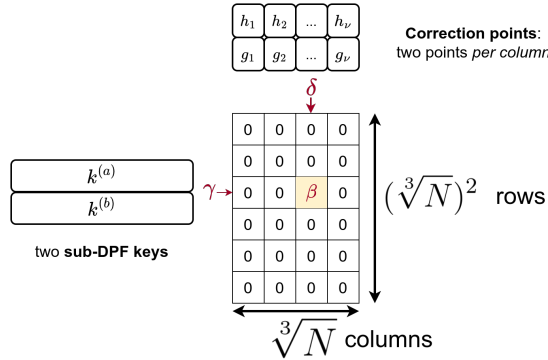


Fig. 1: High-level structure of our DDH-based DPF

Gen: Let $\mathbb{G}$ be a cyclic group of prime order q_0 , with g a generator. We assume that the DDH assumption holds in $\mathbb{G}$. Let $g_\beta = \text{Encode}_{\text{DDH}}(\beta)$ be the encoding of β in $\mathbb{G}$ (possible encodings are discussed in Section 6). Our Gen algorithm starts by sampling one random number $r \xleftarrow{R} \mathbb{F}_{q_0}$ and compute r_{inv} its multiplicative inverse. This multiplicative inverse exists because q_0 is prime.

We then use the sub-DPF to share two secret point functions f_a, f_b such that $f_a(x) = r$ and $f_b(x) = 1$, if $x = \gamma_*$, otherwise 0. This step outputs two sets of DPF keys $\{k_1^{(a)}, \dots, k_p^{(a)}\}$ and $\{k_1^{(b)}, \dots, k_p^{(b)}\}$, with each key of size $O(\sqrt[3]{N})$.

Algorithm 1 DDH-based DPF scheme

-
- 1: Let $\mathbb{G}$ be a cyclic group of prime order q_0 and with g a generator. We assume that the DDH assumption holds in $\mathbb{G}$.
 - 2: Let $(\text{Gen}_{\text{DPF}}^{(*)}, \text{Eval}_{\text{DPF}}^{(*)}, \text{Decode}_{+})$ be the information-theoretic DPF proposed by Bunn et al. [10] (with output in $\mathbb{F}_{q_0}$).
 - 3: Let $\text{Encode}_{\text{DDH}} : \mathcal{Y} \rightarrow \mathbb{G}$ and $\text{Decode}_{\text{DDH}} : \mathbb{G} \rightarrow \mathcal{Y}$ be two functions such that $\text{Encode}_{\text{DDH}}(0) = g^0$ and $\text{Decode}_{\text{DDH}}(\text{Encode}_{\text{DDH}}(y)) = y$ for any $y \in \mathcal{Y}$.
 - 4: **function** $\text{Gen}_{\text{DPF}}(\alpha, \beta, p, m)$
 - 5: Let $g_\beta \leftarrow \text{Encode}_{\text{DDH}}(\beta)$.
 - 6: Let (γ_*, δ_*) be the position of α in a $(\nu)^2 \times \nu$ grid, with $\nu = \lceil \sqrt[3]{N} \rceil$
 - 7: Sample one (non-zero) $r \xleftarrow{R} \mathbb{F}_{q_0}$ and set $r_{\text{inv}} \leftarrow r^{-1}$.
 - 8: Generate two sets of DPF keys:

$$\{k_1^{(a)}, \dots, k_p^{(a)}\} \leftarrow \text{Gen}_{\text{DPF}}^{(*)}(\gamma_*, r, p, m)$$

$$\{k_1^{(b)}, \dots, k_p^{(b)}\} \leftarrow \text{Gen}_{\text{DPF}}^{(*)}(\gamma_*, 1, p, m).$$
 - 9: **for** $\delta \in \{1 \dots \nu\}, \delta \neq \delta_*$ **do**
 - 10: Sample a point g_δ from $\mathbb{G}$ and set $h_\delta \leftarrow g_\delta^{-r_{\text{inv}}}$.
 - 11: Sample a g_{δ_*} from $\mathbb{G}$ and set $h_{\delta_*} \leftarrow g_{\delta_*}^{-r_{\text{inv}}} \cdot g_\beta^{r_{\text{inv}}}$.
 - 12: Set $k_i = (k_i^{(a)} || k_i^{(b)} || g_1 || h_1 || \dots || g_\nu || h_\nu), \forall i \in \{1 \dots p\}$
 - 13: **return** $(k_1, \dots, k_p)$.
 - 14: **function** $\text{Eval}_{\text{DPF}}(k_i, x)$
 - 15: Let (γ', δ') be the position of x in a $(\sqrt[3]{N})^k \times \sqrt[3]{N}$ grid.
 - 16: Parse k_i as $k_i = (k_i^{(a)} || k_i^{(b)} || g_1 || h_1 || \dots || g_\nu || h_\nu)$.
 - 17: Let $\llbracket s_a \rrbracket_i \leftarrow \text{Eval}_{\text{DPF}}^{(*)}(k_i^{(a)}, \gamma'), \llbracket s_b \rrbracket_i \leftarrow \text{Eval}_{\text{DPF}}^{(*)}(k_i^{(b)}, \gamma')$.
 - 18: Let $\llbracket f(x) \rrbracket_i \leftarrow h_{\delta'}^{\llbracket s_a \rrbracket_i} \cdot g_{\delta'}^{\llbracket s_b \rrbracket_i}$.
 - 19: **return** $\llbracket f(x) \rrbracket_i$ $\triangleright \llbracket f(x) \rrbracket_i \in \mathbb{G}$.
 - 20: **function** $\text{Decode}_{\text{DPF}}(\llbracket f(x) \rrbracket_1, \dots, \llbracket f(x) \rrbracket_p)$ **return** $\text{Decode}_{\text{DDH}}(\prod_{i=1}^p \llbracket f(x) \rrbracket_i)$
-

thanks to the sub-DPF. For each column δ , we sample a random generator $g_\delta \xleftarrow{R} \mathbb{G}$. For all $\delta \neq \delta_*$, we set the “correction points” $h_\delta = g_\delta^{r_{\text{inv}}}$. We set the correction points for the column δ_* : $h_{\delta_*} \leftarrow g_{\delta_*}^{-r_{\text{inv}}} \cdot g_\beta^{r_{\text{inv}}}$. Each party i receives a key k_i containing two DPF “sub-keys” $k_i^{(a)}, k_i^{(b)}$ and all corrections points.

Eval: represents x as (γ', δ') and evaluates: $\llbracket s_a \rrbracket_i \leftarrow \text{Eval}_{\text{DPF}}(k_i^{(a)}, \gamma')$ and $\llbracket s_b \rrbracket_i \leftarrow \text{Eval}_{\text{DPF}}(k_i^{(b)}, \gamma')$. Each party obtains $\llbracket f(x) \rrbracket_i = h_{\delta'}^{\llbracket s_a \rrbracket_i} \cdot g_{\delta'}^{\llbracket s_b \rrbracket_i}$.

Algorithm 1 details our DDH-based scheme.

3.2 Asymptotic key size

Figure 1 provides a high-level representation of our DPF keys, with an $O(\sqrt[3]{N} \cdot \binom{p-1}{m} \cdot (\lambda + \log q))$ key size. Let us break down this asymptotic cost.

First, we have two sub-DPF keys. The previous DPFs are defined over the domain $\{1 \dots (\sqrt[3]{N})^2\}$ to “cover all the rows”. We have $(\sqrt[3]{N})^2$ rows and use [10]

as sub-DPF (having an $O(\sqrt{M} \binom{p-1}{m} \log q)$ key size for a domain of size M). Each sub-DPF key is then of size $O(\sqrt[3]{N} \cdot \binom{p-1}{m} \cdot \log q_0)$ ($M = (\sqrt[3]{N})^2$ in our case).

Second, for each column δ , we have $g_\delta, h_\delta \in \mathbb{G}$ (with $q = |\mathbb{G}|$). As we have $\sqrt[3]{N}$ columns, the total size of these elements is $O(\sqrt[3]{N} \cdot \log q)$.

In our scheme, λ is implicitly present because DDH is assumed to be hard in cyclic group $\mathbb{G}$. As $\mathbb{G}$ is of order q_0 , we have $\lambda = O(\log q_0)$.

Supporting other sub-DPFs We use a grid of size $(\sqrt[3]{N})^2 \times \sqrt[3]{N}$ because the sub-DPF we use has an $O(\sqrt{M})$ key size (for a domain size of M). However, using our DDH-based approach, we can replace [10] with any scheme that outputs additive shares in $\mathbb{F}_{q_0}$. Specifically, for any sub-scheme with key size $O(\sqrt[k]{N})$, our method yields a DDH-based variant with key size $O(\sqrt[k+1]{N})$.

In particular, our approach can be applied to the dishonest-majority scheme proposed in [18]. The resulting scheme (combining [18] with our DDH-based approach) is secure against a dishonest majority and has key size $O(\sqrt[3]{N} \cdot p^3 \lambda^3 \cdot (\log q + \lambda))$. As shown in Section 4, the scheme of [18] already has larger keys than sharing the function's truth table. Therefore, extending our scheme using [18] would result in an impractical solution, because [18] is impractical.

Given this, we do not further develop this extension in our work. Instead, we focus our discussions solely on our honest-majority scheme, which, in contrast, provides substantial key size reductions for practical problem sizes.

Since we can reduce key sizes from $O(\sqrt[k]{N})$ to $O(\sqrt[k+1]{N})$, our result raises a key question: can we build a multi-party scheme with logarithmic key size via a recursive application of our scheme? Our DDH-based scheme outputs multiplicative shares (in a DDH group), so it cannot be used recursively.

3.3 Security

In our scheme, the DDH assumption prevents an adversary from distinguishing the correction points, and then prevents them from recovering the column δ_* containing the non-zero from the key. To prove this intuition, we propose to consider the following theorem:

Theorem 1. *Let $\lambda \in \mathbb{N}$, $N, p \in \mathbb{N}$, then $(\text{Gen}_{\text{DPF}}, \text{Eval}_{\text{DPF}}, \text{Decode}_{\text{DPF}})$ as described in Algorithm 1 is an FSS scheme for the family of point functions.*

Assuming that the DDH assumption holds in $\mathbb{G}$ and that the information-theoretic scheme $(\text{Gen}_{\text{DPF}}^{()}, \text{Eval}_{\text{DPF}}^{(*)}, \text{Decode}_+)$ from [10] is a correct and private DPF scheme, then this scheme is correct and private against at most m semi-honest parties with $m < p/2$.*

Proof. Given in Appendix A.

3.4 Comparison with existing works

Table 1 compares existing schemes to ours. We achieve two goals: (1) avoiding exponential factors present in [4,10], and (2) improving the dependency on the

Scheme	Year	Majority	Assumptions	Key size
[4]	2015	Dishonest	PRG	$O(\sqrt{N} \cdot q^{\frac{p-1}{2}} \cdot (\log q + \lambda))$
[12,22]	2015	Dishonest	DDH	$O(\sqrt{N} \cdot (\lambda + \log q))$
PRG-based [10]	2022	Honest	PRG	$O(\sqrt[p]{N} \cdot \sqrt[q]{q^{p^m}} \cdot \binom{p-1}{m} \cdot (\lambda + \log q))$
Info.-Th. [10]	2022	Honest	None	$O(\sqrt{N} \cdot \binom{p-1}{m} \cdot \log q)$
[18]	2025	Dishonest	PRG	$O(\sqrt{N} \cdot p^3 \lambda^3 \cdot (\log q + \lambda))$
[20]	2025	Dishonest	DDH + 2 new hardness assumpt.	$O(\log N \cdot (\log q + \lambda))$
Our scheme	2025	Honest	DDH	$O(\sqrt[p]{N} \cdot \binom{p-1}{m} \cdot (\lambda + \log q))$

Table 1: Comparison of the multi-party DPF schemes

function domain size N (compared to other practical schemes [10,12,18]). We offer an intermediary solution between an $O(\sqrt[p]{N})$ key size with exponential factors [10] and $O(\sqrt{N})$ key sizes without exponential factors [10,12,18].

The scheme of [20] achieves the best known asymptotic key size, namely $O(\log N)$. However, this improvement comes at the cost of relying on two new hardness assumptions whose concrete security is not yet well understood. The work of [20] represents an exciting theoretical advancement, but the lack of concrete security hinders its implementation *for now*. Further research is needed to determine parameter sets that achieve standard security levels (e.g., 128-bit or 256-bit). In contrast, our scheme has $O(\sqrt[p]{N})$ key sizes, but is based solely on the DDH assumption; a *standard and extensively studied cryptographic assumption*.

3.5 Pseudo-random secret sharing

Pseudo-random secret sharing (PRSS) [13] is a protocol reducing the communication cost of secret sharing protocols thanks to pseudo-random seed expansion. As most multi-party schemes [4,10,12] (including ours) output DPF keys containing secret shares, we can reduce their size thanks to PRSS.

In particular, we rely on the simplest form of PRSS applied to additive secret shares. Additive secret sharing on a secret s normally outputs p shares $\{\llbracket s \rrbracket_1 \dots \llbracket s \rrbracket_p\}$, such that $\sum_i \llbracket s \rrbracket_i = s$. With PRSS, the secret holder generates $p-1$ random seeds $r_1, \dots, r_{p-1}$ and a share $\llbracket s \rrbracket_p$, such that $\llbracket s \rrbracket_p + \sum_{i=1}^{p-1} G(r_i) = s$ (with G a PRG). One shareholder receives $\llbracket s \rrbracket_p$ and the other $p-1$ parties receive a random seed r_i .

The random seeds can be reused to share multiple values. To share a vector of n values, the secret holder sends $p-1$ seeds and a single vector of n shares

(vs. p vector of n shares in classic secret sharing). If the MPC protocol allows the secret holder to be a shareholder, they can keep the vector of shares and send only random seeds to the other shareholders.

This technique amortizes the communication costs (i.e., key size) when the protocols require sharing large vectors. Since all practical schemes [10,12] are compatible with PRSS, Section 4 compares the key sizes using PRSS.

4 Key size benchmark

We compare the exact key size of our DPF scheme to those of existing schemes. Our experiments report the total key size instead of the individual key size. This metric takes into account the key size amortization provided by PRSS. For this benchmark, we implement our scheme using elliptic curves $E(\mathbb{F}_q)$, as they provide prime-order cyclic groups in which the DDH is known to be hard. More specifically, we use the curve P-256.

Exponential factors in existing works Figure 2 illustrates the exponential factor of the PRG-based multi-party DPF [4,10]. Figure 2a represents their key size in function of the output bit length (i.e., $\log_2 q$ for an output in $\mathbb{F}_q$). While this problem was barely discussed in existing works [4,10], Figure 2 shows that it makes these techniques purely unusable. For only 12-bit moduli, the DPF of [4] has a key size of 10^{18} bits and the PRG-based DPF of [10] has a key size of 10^{38} .

Figure 2 includes a curve “Trivial scheme”, corresponding to the most trivial DPF implementation: sharing the truth table of the function. This trivial baseline is much better than these PRG-based solutions for prime fields other than $\mathbb{F}_2$. While other works [3,22] warned about this potential scalability issue, our work provides the first evidence of their impracticability in $\mathbb{F}_q$.

In comparison, we can barely see on Figure 2a the key size difference between the rest of the DPF schemes [10,12, Ours]. Note that Figure 2 includes curves for the DDH-based schemes [12, Ours]. Such schemes are implemented with a fixed elliptic curve P-256, so the output is of fixed size (256 bits). Thus, their key sizes are constant on this figure.

Recently, Boyle [3] proposed an optimization based on the Chinese Remainder Theorem (CRT) to optimize the key sizes of PRG-based schemes [4,10] for composite moduli. Thanks to this CRT trick, we can replace the exponential factor q^p with a sum of smaller factors $\sum_i q_i^p$ (with q_i the prime factors of the composite modulus).

Figure 2b represents the key sizes for varying composite moduli, but the figure is hard to read, because this trick is sensitive to the prime decomposition. Two numbers m and $m + 1$ can have completely different prime factors. To provide more readable results, Figure 2c represents the key sizes for varying primorial moduli. A primorial is a composite number whose prime factors are the n smallest primes. Such numbers are the best case scenario for the CRT trick because they provide the smallest factors possible. On Figure 2c, we observe that the trick significantly improves the key size of the PRG-based schemes, but they remain way above the trivial scheme for any modulus above 210.

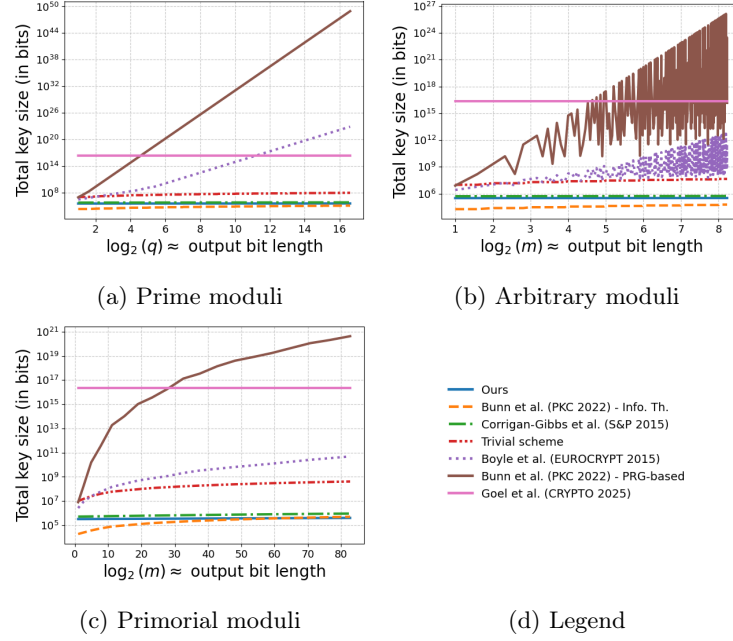


Fig. 2: DPF key sizes for varying moduli. Parameters: $p = 5$ parties, $N = 10^6$.

Finally, Figure 2 provides a key insight into [18]. Note that their key size may seem constant in q on Figure 2, but it is not; their key size is simply dominated by factors independent of q . While their work replaces the exponential factor present in [4] with a polynomial factor of $p^3 \lambda^3$, we demonstrate that, for realistic problem sizes, this polynomial term actually exceeds q^p . As a result, the asymptotic improvement offered by [18] does not translate into practical efficiency: the key size remains significantly larger than that of the trivial scheme. In other words, although [18] presents a notable theoretical advance by eliminating the exponential term of [4], the resulting scheme remains impractical; performing worse than the trivial DPF scheme.

Since PRG-based schemes [4,10,18] are impractical for moduli larger than 210 (even with the CRT optimization of [3]), we exclude them from our other experiments. This allows us to observe precisely the key size improvements achieved by our scheme when compared to practical DPF schemes [10,12].

Varying function domain Figure 3a compares the key sizes in function of the domain sizes; varying from 10^2 to 10^{10} . Zyskind et al. [30] used the same problem sizes to benchmark three-party DPF schemes.

Our key size is the smallest for any domain size above 10^3 . When $N = 10^6$, our key size is 3 times smaller. When $N = 10^9$, our key size is 10 times smaller.

Our key size is higher than those of [10,12] only when N is below 10^3 . To put this into perspective, Figure 3a also shows that the trivial DPF scheme becomes

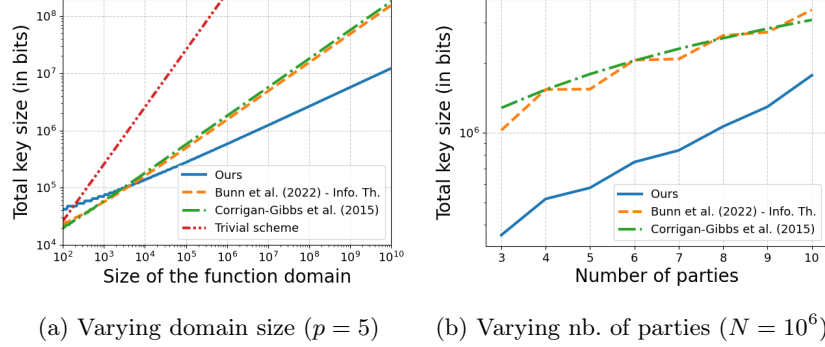


Fig. 3: Key sizes for varying domain sizes and number of parties.

the most efficient for smaller domain sizes (below 10^2). Thus, the advantage provided [10,12] only holds for particularly small function domains and the trivial DPF could even be preferable in such cases.

Varying number of parties Figure 3b compares the key sizes in function of the number of parties; varying from 3 to 10. We observe that Corrigan-Gibbs et al. [12] has a better scaling than our scheme, but our key size remains smaller.

The growth of our key size is not as smooth as the growth of the scheme by [12]. For example, the growth between 3 and 4 parties is steeper than between 4 and 5. This phenomenon is caused by the honest-majority assumption: (contrary to [12]) our total key size is correlated to the number of honest parties. Under the honest-majority assumption, 4-party and 5-party setups have exactly the same number of honest parties (i.e., 3). The same phenomenon is present for [10].

Impact of PRSS Figures 3a and 3b represents the key sizes using PRSS (for all schemes), but it is interesting to study the impact of PRSS on the key size. Hence, Figure 4 illustrates the impact of PRSS on our key size. This figure shows that PRSS *approximately* reduces the key size by a factor 5 (i.e., number of parties in this experiment).

We observe some “step” patterns on the curves (especially on Figure 4 for domain sizes below 10^4). These patterns are due to the grid representation in the multi-party DPF schemes. For example, our scheme represents any point x in a grid of size $\sqrt[3]{N}^2 \times \sqrt[3]{N}$. However, $\sqrt[3]{N}$ is unlikely to be an integer, so our algorithms round these values; inducing the thresholds visible in all techniques (if we zoom in on the plots).

5 DDH-based Distributed Comparison Function

This section adapts our DPF scheme to comparison functions. However, the literature has described fewer DCF schemes than DPF. Indeed, only [4,22] have

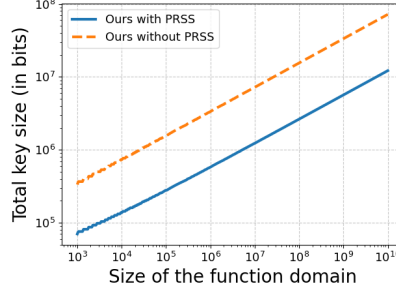


Fig. 4: Impact of the pseudo-random secret-sharing on our scheme.
Parameters: 5 parties.

described multiparty DCFs, which provides weak baselines compared to DPF. Thus, Appendix B adapts the DPF schemes of [10] to DCF in order to compare our DCF to challenging baselines.

5.1 Scheme

We build our DDH-based DCF upon the information-theoretic DPF of [10] and the DCF adapted from [10] (see Appendix B). The main change compared to our DPF scheme is that we require both a sub-DCF and a sub-DPF. Algorithm 2 details our scheme (with the changes compared to our DPF in [blue](#)).

Figure 1 provides a high-level representation of our DCF key.

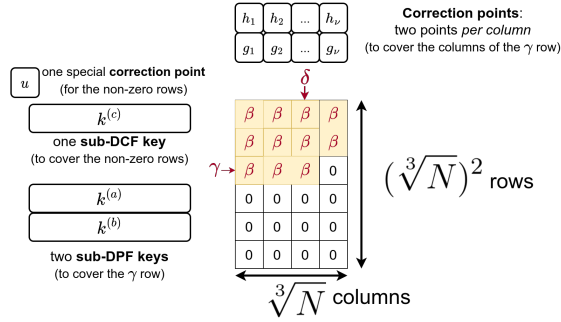


Fig. 5: High-level structure of our DDH-based DCF

To understand their respective roles, let us represent the function domain as a grid. On the one hand, the DCF covers all rows containing only non-zeros values (i.e., β). On the other hand, the DPF enables to select the row γ_* containing a segment of β and a segment of 0. To complete our extension, we update the

Algorithm 2 DDH-based DCF scheme (changes compared to our DPF in blue)

-
- 1: Let $\mathbb{G}$ be a cyclic group of prime order q_0 and with g a generator.
 - 2: Let $(\text{Gen}_{\text{DPF}}^{(*)}, \text{Eval}_{\text{DPF}}^{(*)}, \text{Decode}_+)$ be the information-theoretic DPF of Bunn et al. [10] and $(\text{Gen}_{\text{DCF}}^{(*)}, \text{Eval}_{\text{DCF}}^{(*)}, \text{Decode}_+)$ be the DCF adapted from it.
 - 3: Let $\text{Encode}_{\text{DDH}} : \mathcal{Y} \rightarrow \mathbb{G}$ and $\text{Decode}_{\text{DDH}} : \mathbb{G} \rightarrow \mathcal{Y}$ be two functions such that $\text{Encode}_{\text{DDH}}(0) = g^0$ and $\text{Decode}_{\text{DDH}}(\text{Encode}_{\text{DDH}}(y)) = y$ for any $y \in \mathcal{Y}$.
 - 4: **function** $\text{Gen}_{\text{DPF}}(\alpha, \beta, p, m)$
 - 5: Let $g_\beta \leftarrow \text{Encode}_{\text{DDH}}(\beta)$.
 - 6: Let (γ_*, δ_*) be the position of α in a $(\nu)^k \times \nu$ grid, with $\nu = \lceil \sqrt[3]{N} \rceil$.
 - 7: Sample two (non-zero) $r, s \xleftarrow{R} \mathbb{F}_{q_0}$ and set $r_{\text{inv}} \leftarrow r^{-1}$ and $s_{\text{inv}} \leftarrow s^{-1}$.
 - 8: Generate two sets of DPF keys and one of DCF keys:

$$\{k_1^{(a)}, \dots, k_p^{(a)}\} \leftarrow \text{Gen}_{\text{DPF}}^{(*)}(\gamma_*, r, p, m)$$

$$\{k_1^{(b)}, \dots, k_p^{(b)}\} \leftarrow \text{Gen}_{\text{DPF}}^{(*)}(\gamma_*, 1, p, m)$$

$$\{k_1^{(c)}, \dots, k_p^{(c)}\} \leftarrow \text{Gen}_{\text{DCF}}^{(*)}(\gamma_* - 1, r_c, p, m).$$
 - 9: **for** $\delta > \delta_*$ **do** Sample a point g_δ from $\mathbb{G}$ and set $h_\delta \leftarrow g_\delta^{-r_{\text{inv}}}$.
 - 10: **for** $\delta \leq \delta_*$ **do** Sample a g_δ from $\mathbb{G}$ and set $h_\delta \leftarrow g_\delta^{-r_{\text{inv}}} \cdot g_\beta^{r_{\text{inv}}}$.
 - 11: Let $u \leftarrow g_\beta^{s_{\text{inv}}}$.
 - 12: Set $k_i = (k_i^{(a)} || k_i^{(b)} || g_1 || h_1 || \dots || g_\nu || h_\nu || u), \forall i \in \{1 \dots p\}$
 - 13: **return** $(k_1, \dots, k_p)$.
 - 14: **function** $\text{Eval}_{\text{DPF}}(k_i, x)$
 - 15: Let (γ', δ') be the position of x in a $(\sqrt[3]{N})^2 \times \sqrt[3]{N}$ grid.
 - 16: Parse $k_i = (k_i^{(a)} || k_i^{(b)} || g_1 || h_1 || \dots || g_\nu || h_\nu || u)$.
 - 17: Let $\llbracket s_a \rrbracket_i \leftarrow \text{Eval}_{\text{DPF}}^{(*)}(k_i^{(a)}, \gamma'), \llbracket s_b \rrbracket_i \leftarrow \text{Eval}_{\text{DPF}}^{(*)}(k_i^{(b)}, \gamma')$
 - 18: Let $\llbracket s_c \rrbracket_i \leftarrow \text{Eval}_{\text{DCF}}^{(*)}(k_i^{(c)}, \gamma')$
 - 19: Let $\llbracket f(x) \rrbracket_i \leftarrow h_{\delta'}^{\llbracket s_a \rrbracket_i} \cdot g_{\delta'}^{\llbracket s_b \rrbracket_i} \cdot u^{\llbracket s_c \rrbracket_i}$.
 - 20: **return** $\llbracket f(x) \rrbracket_i$ $\triangleright \llbracket f(x) \rrbracket_i \in \mathbb{G}$.
 - 21: **function** $\text{Decode}_{\text{DPF}}(\llbracket f(x) \rrbracket_1, \dots, \llbracket f(x) \rrbracket_p)$ **return** $\text{Decode}_{\text{DDH}}(\prod_{i=1}^p \llbracket f(x) \rrbracket_i)$
-

definition of the “correction points”, so that the γ_* row contains β until the column δ_* and 0 after.

This scheme is a simple adaptation of our DPF; adding a (secure) sub-DCF and updating the correction points. Thus, its security proof relies on the same arguments as our DPF scheme (see Appendix A).

5.2 Comparison

Table 2 compares our DCF schemes to the existing schemes. The DCF schemes adapted from [10] have respectively $O(\sqrt[4]{N} \cdot \sqrt{q^{p^m}} \cdot \binom{p-1}{m} \cdot (\lambda + \log q))$ key size for the PRG-based and $O(\sqrt{N} \cdot \binom{p-1}{m} \cdot \log q)$ key size for the information-theoretic. Finally, our DDH-based scheme has a key size of $O(\sqrt[3]{N} \cdot \binom{p}{m+1} \cdot \log q)$.

Scheme	Majority	Assumption	Key size
[4]	Dishonest	PRG	$O(\sqrt{N} \cdot q^{\frac{p-1}{2}} \cdot (\lambda + \log q))$
[22]	Dishonest	DDH + Trusted decoder	$O(\sqrt{N}(2^{-\frac{p-1}{2}} \cdot \lambda + 2^{\frac{p-1}{2}} \cdot \log q))$
Our adaptation PRG-based [10]	Honest	PRG	$O(\sqrt[4]{N} \cdot \sqrt{q^{p^m}} \cdot \binom{p-1}{m} \cdot (\lambda + \log q))$
Our adaptation Info.-Th. [10]	Honest	None	$O(\sqrt{N} \cdot \binom{p-1}{m} \cdot \log q)$
Our scheme	Honest	DDH	$O(\sqrt[3]{N} \cdot \binom{p-1}{m} \cdot (\lambda + \log q))$

Table 2: Comparison of the multi-party DCF schemes

We obtain the same conclusions as in multi-party DPF: our scheme provides an intermediary solution between efficient schemes [22,10] with $O(\sqrt{N})$ key size and an impractical PRG-based scheme [10] with $O(\sqrt[4]{N}q^p)$ key size.

6 DDH-based secret encodings and their applications

Our schemes (as well as other DDH-based schemes) need to encode/decode the secret value β in the cyclic group $\mathbb{G}$. This encoding and decoding steps are essential as FSS applications usually involve real-valued or integer secrets that must be mapped to $\mathbb{G}$. Existing papers [12,22] have been vague on the exact way to encode and handle secret in DDH-based schemes.

To have a discussion as concrete as possible, this section uses elliptic curves as cyclic group; $\mathbb{G} = E(\mathbb{F}_q)$. Elliptic curves provide (DDH-hard) cyclic group with prime orders. They were notably used by [12] as they provide smaller key sizes than other DDH-hard cyclic groups.

Contrary to other DPF schemes [4,10], DDH-based schemes use multiplication to decode the secret shares. However, using elliptic curves, the multiplication in the cyclic group corresponds to an addition of elliptic curve points. Even though elliptic curves provide a form of additive decoding, this decoding does not provide the same properties as linear share decoding in $\mathbb{F}_q$.

Thus, this section presents two possible DDH-based secret encoding. For each encoding, we present its properties and use cases.

6.1 Share compressibility

Boyle et al. [4] identified some desirable share properties, in particular: compressibility. According to them, a scheme has compressible shares if we can combine

the shares “in a meaningful way” without communication between the shareholders. They focused on the aggregation of multiple shares: $\llbracket f(x_1) \rrbracket \oplus \llbracket f(x_2) \rrbracket = \llbracket f(x_1) + f(x_2) \rrbracket$. Many FSS applications (e.g., PIR [17] or private histograms [2]) require a form of compressibility.

Works like [4,10] satisfy this property thanks to their additive secret-sharing in $\mathbb{F}_q$. However, “DDH-based” encoding can require a non-linear share decoding, that does not necessarily guarantee compressibility. The next subsections present two DDH-based secret encodings, discuss their relation to compressibility, and list some applications.

6.2 Point-based secret encoding

An elliptic curve point P is defined using the coordinates (x, y) . Using the curve formula, we can recover y for any given x .

Point encoding consists in finding $P_\beta = (\beta, y_\beta)$, with $y_\beta \geq 0$. The shares are randomly sampled points $P_1, \dots, P_p \in E(\mathbb{F}_q)$ such that $\sum_i P_i = P_\beta$. The share decoding requires summing the p points and then extracting the first coordinate of the sum result.

Discussion However, this encoding prevents a general share compressibility because we cannot sum two encoded secrets. This constraint comes from the properties of the elliptic curve addition. Consider two secrets $\beta_1, \beta_2 \in \mathbb{F}_q$ and their respective point-based encodings P_{β_1}, P_{β_2} , let $P_* = P_{\beta_1} + P_{\beta_2} = (x_*, y_*)$. In the general case, we have $x_* \neq \beta_1 + \beta_2$.

Nevertheless, there is an exception to this impossible addition: if one of the shared values is 0. By convention, we encode zero values using the point at infinity $\mathcal{O}$. Hence, we have $P_\beta + P_0 = P_\beta + \mathcal{O} = P_\beta$. Thus, point-based encoding provides a limited compressibility: we can sum multiple function shares only if, at most one of them is not null.

DPF Use case Anonymous broadcasters enable several parties to broadcast a message without revealing the message origin. These systems often take the form of public bulletin board in which the writers are anonymous. In recent years, DPFs have become valuable primitives to build efficient anonymous broadcasters like in Riposte [12] and Spectrum [25].

Their DPF-based protocols have the following structure: (1) the servers initialize a large shared bulletin board, (2) each sender i picks at random position α_i in the bulletin board, (3) each sender i uses their message m_i to share a secret point function f_i such that $f_i(x) = m_i$ if $x = \alpha_i$, 0 otherwise, (4) for each shared function $\llbracket f_i \rrbracket$, the servers obtain a shared vector $\llbracket v_i \rrbracket$ such that $\llbracket v_i[j] \rrbracket = \llbracket f_i(j) \rrbracket$, (5) they sum all the vectors v_i with the bulletin board, and reveal the final bulletin board. In other words, the DPF is used as an oblivious write operation on a secret-shared bulletin board.

If at most one sender picks an index, the limited compressibility is sufficient. Spectrum and Riposte proposed solutions to avoid two senders from choosing the same index.

DCF Use case Many recent works [19,22,26] considered FSS to optimize non-linear operations in privacy-preserving machine learning. For instance, Kumar et al. [22] used DCF to perform efficient and secure ReLU operations. This last protocol does not require aggregating multiple shared secrets, so compressibility is not necessary; which makes point-based encoding perfectly acceptable.

6.3 Exponent-based secret encoding

This second encoding represents the secret value as an exponent: $P_\beta = \beta \cdot P$, with P , a generator of the curve. The shares are randomly sampled points $P_1, \dots, P_p \in E(\mathbb{F}_q)$ such that $\sum_i P_i = P_\beta$; it requires sampling p exponents $(s_1, \dots, s_p)$ such that $\sum s_i = \beta$. The share decoding consists in summing p elliptic curve points and then computing the discrete logarithm. While this operation is hard in a general case, it is possible if we assume a bounded β (e.g., $\beta < 10^6$). This assumption is realistic in some applications such as private statistics.

Discussion This encoding provides compressibility because we can sum two encoded secrets: $P_{\beta_1} + P_{\beta_2} = \beta_1 \cdot P + \beta_2 \cdot P = (\beta_1 + \beta_2) \cdot P = P_{\beta_1 + \beta_2}$. However, it has one drawback: the secret must be small, otherwise the decoding is impractical.

DPF Use case Some works [2,24] have used two-party DPFs to build private histograms. In this application, a group of data owners wants to compute the histogram of a certain property (e.g., salary), but each individual wants to keep their value private.

DPF schemes provides a straightforward solution: (1) each individual uses their private value $a_i \in \{1 \dots N\}$ to share a secret point function f_i with $f_i(a_i) = 1$, (2) from each shared function $\llbracket f_i \rrbracket$, the servers can deduce a shared vector $\llbracket v_i \rrbracket$ such that $\llbracket v_i[j] \rrbracket = \llbracket f_i(j) \rrbracket$, (3) The servers aggregate the shared vectors $\llbracket v_i \rrbracket$ and decode the resulting histogram.

This private histogram protocol requires share compressibility because each bin of the histogram usually contains the sum of multiple inputs. Exponent-based encoding is then our only option. The main drawback of exponent-based encoding is that the secret value must be small enough to compute a discrete logarithm, but this assumption is realistic in histogram-making because histogram values number are usually bounded.

DCF Use case Barczewski et al. [1] recently proposed using DCF to estimate empirical cumulative distribution functions (EDCF) on sensitive data. These statistics are essential notably to evaluate the performances of ML models or to perform some statistical tests [1]. As ECDF are related to histograms, they require the same share compressibility properties. Thus, exponent-based encoding is necessary for such applications, assuming the secret is sufficiently small. Like in private histograms, such a bound usually exists in practice.

7 Conclusion

Our work presented a DDH-based approach to build optimized multi-party FSS schemes from existing schemes. In particular, it results into practical schemes with $O(\sqrt[3]{N})$ key sizes instead of $O(\sqrt{N})$. We applied our technique both on DPF (i.e., FSS for point functions) and DCF (i.e., FSS for comparison functions). Finally, our benchmark highlighted key size reductions up to a factor 10 compared to state-of-the-art schemes on realistic problem sizes.

Acknowledgments. This work was supported by the Netherlands Organization for Scientific Research (De Nederlandse Organisatie voor Wetenschappelijk Onderzoek) under NWO:SHARE project [CS.011].

A Appendix: Proof of Theorem 1

Proof. Correctness: We represent any input x as (γ, δ) and α as (γ_*, δ_*) . We need to study the FSS output in three cases: (1) $\gamma \neq \gamma_*$, (2) $\gamma = \gamma_*$ and $\delta \neq \delta_*$, and (3) $\gamma = \gamma_*$ and $\delta = \delta_*$.

If $\gamma \neq \gamma_*$, each party i holds $\llbracket s_a \rrbracket_i, \llbracket s_b \rrbracket_i$ (obtained using their sub-DPF keys), such that $\sum \llbracket s_a \rrbracket_i = \sum \llbracket s_b \rrbracket_i = 0$ (because the sub-DPF is correct by assumption and outputs additive shares). For any δ :

$$\begin{aligned} \prod_i \llbracket f(x) \rrbracket_i &= \prod_i (h_\delta^{\llbracket s_a \rrbracket_i} \cdot g_\delta^{\llbracket s_b \rrbracket_i}) = h_\delta^{\sum_i \llbracket s_a \rrbracket_i} \cdot g_\delta^{\sum_i \llbracket s_b \rrbracket_i} = h_\delta^0 \cdot g_\delta^0 = g^0 \\ \Rightarrow \text{Decode}_{\text{DPF}}(\text{Eval}_{\text{DPF}}(k_1, x), \dots, \text{Eval}_{\text{DPF}}(k_p, x)) &= \text{Decode}_{\text{DDH}}(g^0) = 0 \end{aligned}$$

If $\gamma = \gamma_*$ and $\delta \neq \delta_*$, each party i holds $\llbracket s_a \rrbracket_i, \llbracket s_b \rrbracket_i$, such that $\sum_i \llbracket s_a \rrbracket_i = r$ and $\sum_i \llbracket s_b \rrbracket_i = 1$. The parties obtain:

$$\begin{aligned} \prod_i \llbracket f(x) \rrbracket_i &= \prod_i (h_\delta^{\llbracket s_a \rrbracket_i} \cdot g_\delta^{\llbracket s_b \rrbracket_i}) = h_\delta^{\sum_i \llbracket s_a \rrbracket_i} \cdot g_\delta^{\sum_i \llbracket s_b \rrbracket_i} \\ &= g_\delta^{-r_{\text{inv}} \cdot r} \cdot g_\delta = g_\delta^{-1} \cdot g_\delta = g^0 \\ \Rightarrow \text{Decode}_{\text{DPF}}(\text{Eval}_{\text{DPF}}(k_1, x), \dots, \text{Eval}_{\text{DPF}}(k_p, x)) &= \text{Decode}_{\text{DDH}}(g^0) = 0 \end{aligned}$$

If $\gamma = \gamma_*$ and $\delta = \delta_*$, we also have $\sum_i \llbracket s_a \rrbracket_i = r$ and $\sum_i \llbracket s_b \rrbracket_i = 1$, but h_δ is now equal to $(g_\delta^{-r_{\text{inv}}} \cdot g_\beta^{r_{\text{inv}}})$. It implies:

$$\begin{aligned} \prod_i \llbracket f(x) \rrbracket_i &= \prod_i (h_\delta^{\llbracket s_a \rrbracket_i} \cdot g_\delta^{\llbracket s_b \rrbracket_i}) = h_\delta^{\sum_i \llbracket s_a \rrbracket_i} \cdot g_\delta^{\sum_i \llbracket s_b \rrbracket_i} \\ &= g_\delta^{-r_{\text{inv}} \cdot r} \cdot g_\beta^{r_{\text{inv}} \cdot r} \cdot g_\delta = g_\beta \\ \Rightarrow \text{Decode}_{\text{DPF}}(\text{Eval}_{\text{DPF}}(k_1, x), \dots, \text{Eval}_{\text{DPF}}(k_p, x)) &= \text{Decode}_{\text{DDH}}(g_\beta) = \beta \end{aligned}$$

Privacy: This proof will rely on a construction also used in [12] to prove the security of their DDH-based DPF: Seed-Homomorphic PseudoRandom Generators (SH-PRG). A PRG G is seed-homomorphic if it satisfies the following

property: for any seeds s_1, s_2 , $G(s_1 + s_2) = G(s_1) \oplus G(s_2)$. In particular, Corrigan-Gibbs et al. [12] rely on DDH-based SH-PRG G_{DDH} : given L randomly sampled public parameters $(g_1, \dots, g_L) \in \mathbb{G}^L$, $G_{\text{DDH}}(s) = (g_1^s, \dots, g_L^s)$ for any seed s . SH-PRGs naturally inherit all properties of a PRG, notably the fact that the output is computationally indistinguishable from true randomness if the seed is unknown to the adversary.

Recall the two assumptions of Theorem 1: (1) the DDH is hard, and (2) the sub-DPF scheme is private and correct. To prove the privacy (see Definition 3), we must show that (for every set of corrupted parties $S \subseteq \{1 \dots p\}$ of size m) there exists an efficient simulator that, for any input functions, outputs samples from a distribution that is computationally indistinguishable from the distribution of the DPF keys.

Remind that each key k_i contains the following elements: two sub-DPF keys $(k_i^{(a)}, k_i^{(b)})$ and pairs of “correction points” $(h_1, g_1), \dots, (h_\nu, g_\nu)$

By assumption, the sub-DPF scheme $(\text{Gen}_{\text{DPF}}^{(*)}, \text{Eval}_{\text{DPF}}^{(*)}, \text{Decode}_+)$ is secure, so there exists an efficient simulator to simulate the sub-DPF keys $(k_i^{(a)}, k_i^{(b)})$ (for all $i \in \{1 \dots p\}$).

To simulate the correction points, for each $\delta \in \{1 \dots \nu\}$, the simulator samples a random generator $\tilde{g}_\delta \in \mathbb{G}$ and another random element $\tilde{h}_\delta \in \mathbb{G}$. Note that g_δ is also sampled randomly during the key generation, so the distribution of g_δ is indistinguishable from the distribution of $\tilde{g}_\delta$.

Finally, we have to prove that the adversary cannot distinguish h_δ from $\tilde{h}_\delta$. To prove this statement, we will consider successively two scenarios: (1) the adversary knows (δ_*, β) and (2) the adversary does not know the pair.

Let us first assume that the adversary knows (δ_*, β) . Note that we can rewrite the tuple $(h_1, \dots, h_{\delta_*}, \dots, h_\nu)$ as $((g_1^{-1})^{r_{\text{inv}}}, \dots, (g_{\delta_*}^{-1} g_\beta)^{r_{\text{inv}}}, \dots, (g_\nu^{-1})^{r_{\text{inv}}})$. As $g_1, \dots, g_\nu$ and g_β are known to the attacker $((g_1^{-1}), \dots, (g_{\delta_*}^{-1} g_\beta), \dots, (g_\nu^{-1}))$, these values can be interpreted as the public parameters of a DDH-based SH-PRG. The only condition on the public parameters is that they must be independently sampled generators. This condition is trivially satisfied for all g_δ with $\delta \neq \delta_*$, and we can observe that g_{δ_*} is independent of g_β so $(g_{\delta_*}^{-1} g_\beta)$ is indistinguishable from random.

Let G_{DDH}^* then be a DDH-based SH-PRG with $((g_1^{-1}), \dots, (g_{\delta_*}^{-1} g_\beta), \dots, (g_\nu^{-1}))$ as public parameters. We have:

$$(h_1, \dots, h_\nu) = ((g_1^{-1})^{r_{\text{inv}}}, \dots, (g_{\delta_*}^{-1} g_\beta)^{r_{\text{inv}}}, \dots, (g_\nu^{-1})^{r_{\text{inv}}}) = G_{\text{DDH}}^*(r_{\text{inv}})$$

In other words, $(h_1, \dots, h_\nu)$ is the output of G_{DDH}^* on the seed r_{inv} . The seed r_{inv} is unknown to the adversary, so they cannot distinguish the real $(h_1, \dots, h_\nu)$ from the randomly sampled $(\tilde{h}_1, \dots, \tilde{h}_\nu)$ outputted by the simulator [12].

Let us now assume that the adversary does not know (δ_*, β) . If the adversary were able to distinguish h_δ from $\tilde{h}_\delta$ without this knowledge, we could trivially build a distinguisher for an adversary knowing (δ_*, β) . Since the h_δ can be formulated as the output of a DDH-based SH-PRG, this hypothetical distinguisher would break the DDH assumption (upon which the SH-PRG is built). By as-

sumption, DDH is hard, so this adversary cannot distinguish real h_δ from the simulator output $\widetilde{h_\delta}$.

B Appendix: Adapting Bunn et al. [10] to DCF

Bunn et al. [10] presented a generic technique to build more efficient honest-majority DPF schemes from dishonest-majority schemes. They applied their technique on two existing schemes: the trivial DPF (i.e., a secret-shared truth table) and on the PRG-based scheme from [4].

While their constructions provides some of the best solutions in DPF (see Table 1), they did not adapt them to comparison functions. This leaves the literature with only weak DCF baselines [4,22]. Thus, we propose to extend their work and build a DCF following the same intuition as their DPF.

Their DPF schemes To understand our adaptation, it is first important to understand their initial DPF. Their main intuition is to represent a point function f as the product of two point functions f_a and f_b (defined over small domains): $f(x) = f_a(\gamma) \times f_b(\delta)$ (with (γ, δ) the representation of x in a $(\sqrt{N} \times \sqrt{N})$ grid). Each function is shared using an existing DPF scheme. However, additive secret shares cannot be multiplied without communications.

Instead of using additive secret sharing, Bunn et al. [10] used replicated secret sharing: each party receives multiple DPF keys. Under the honest-majority assumption, the shareholders can perform one offline multiplication on values shared via replicated secret sharing.

To sum up, Bunn et al. [10] used replicated secret-sharing and the honest-majority assumption to build a technique reducing the key size of existing dishonest-majority schemes from $O(\sqrt[k]{N})$ to $O(\sqrt[2k]{N})$. As reported in Table 1, this technique leads to a PRG-based DPF with $O(\sqrt[4]{N})$ key size and an information-theoretic DPF with $O(\sqrt{N})$ key size.

Our adapted DCF While point functions are the product of two point functions, we can decompose comparison functions using three sub-functions (defined over smaller domains): one point function f_a and two comparison functions f_b, f_c . A comparison function f can be expressed as $f(x) = f_a(\gamma) \times f_b(\delta) + f_c(\gamma)$ with

- $f(x) = \beta$ if $x \leq \alpha$, 0 otherwise; with $x \in \{1 \dots N\}$ and $\alpha = \gamma_* \times \lceil \sqrt{N} \rceil + \delta_*$.
- $f_a(\gamma) = \beta$ if $\gamma = \gamma_*$, 0 otherwise; with $\gamma \in \{1 \dots \lceil \sqrt{N} \rceil\}$.
- $f_b(\delta) = 1$ if $\delta \leq \delta_*$, 0 otherwise; with $\delta \in \{1 \dots \lceil \sqrt{N} \rceil\}$.
- $f_c(\gamma) = \beta$ if $\gamma < \gamma_*$, 0 otherwise; with $\gamma \in \{1 \dots \lceil \sqrt{N} \rceil\}$.

The multiplication of f_a and f_b enables to represent the row γ_* containing a segment of β values and a segment of 0 values. Furthermore, f_c covers all the rows before γ_* full of β values.

Like in [10], we rely on honest-majority and replicated secret sharing to perform the multiplication of f_a and f_b . Due to space limitations, we cannot

Algorithm 3 DCF adapted from the DPF of [10] (changes are in blue)

- 1: Let $(\text{Gen}_{\text{DPF}}^{(*)}, \text{Eval}_{\text{DPF}}^{(*)}, \text{Decode}_+)$ be a $(p-1, p)$ -secure DPF scheme with $O(\sqrt[p]{M})$ key size for a function domain of size M and $(\text{Gen}_{\text{DCF}}^{(*)}, \text{Eval}_{\text{DCF}}^{(*)}, \text{Decode}_+)$ be a $(p-1, p)$ -secure DCF scheme with $O(\sqrt[p]{M})$ key size.
 - 2: Let S be the *sorted* list of all possible combinations of $(p-m)$ elements from $\{1 \dots p\}$; it contains $C = \binom{p}{p-m}$ sets.
 - 3: **function** $\text{Gen}_{\text{DCF}}(\alpha, \beta, p, m)$
 - 4: Let (γ_*, δ_*) be the position of α in an $(\sqrt{N} \times \sqrt{N})$ grid.
 - 5: $\{k_1^{(a)}, \dots, k_C^{(a)}\} \leftarrow \text{Gen}_{\text{DPF}}^{(*)}(\gamma_*, \beta, C, C-1)$.
 - 6: $\{k_1^{(b)}, \dots, k_C^{(b)}\} \leftarrow \text{Gen}_{\text{DCF}}^{(*)}(\delta_*, 1, C, C-1)$
 - 7: $\{k_1^{(c)}, \dots, k_C^{(c)}\} \leftarrow \text{Gen}_{\text{DCF}}^{(*)}(\gamma_* - 1, \beta, C, C-1)$
 - 8: **for** $i \in \{1 \dots p\}$ **do** initialize k_i to an empty bit string.
 - 9: **for** $i \in \{1 \dots p\}, j \in \{1 \dots C\}$ **do**
 - 10: **if** $i \in S[j]$ **then** $k_i \leftarrow k_i || (k_j^{(a)}, k_j^{(b)}, k_j^{(c)})$.
 - 11: **else** $k_i \leftarrow k_i || (\perp, \perp, \perp)$.
 - 12: **return** $(k_1, \dots, k_p)$
 - 13: **function** $\text{Eval}_{\text{DCF}}(k_i, x)$
 - 14: Let (γ', δ') be the position of x in a $\sqrt{N} \times \sqrt{N}$ grid.
 - 15: Parse k_i as $k_i = (k_1^{(a)}, k_1^{(b)}, k_1^{(c)}) || \dots || (k_C^{(a)}, k_C^{(b)}, k_C^{(c)})$.
 - 16: Initialize $\text{res} \leftarrow 0$.
 - 17: **for** $j \in \{1 \dots C\}$ **do**
 - 18: **if** $i == \min S[j]$ **then** ▷ Ensures that each share is only added once.
 - 19: $\text{res} \leftarrow \text{res} + \text{Eval}_{\text{DCF}}^{(*)}(k_j^{(c)}, \gamma')$
 - 20: **for** $j \in \{1 \dots C\}, j' \in \{1 \dots C\}$ **do**
 - 21: **if** $i == \min(S[j] \cap S[j'])$ **then** ▷ Same as Line 18.
 - 22: $\text{res} \leftarrow \text{res} + \text{Eval}_{\text{DPF}}^{(*)}(k_j^{(a)}, \gamma') \times \text{Eval}_{\text{DCF}}^{(*)}(k_j^{(b)}, \delta')$.
 - 23: **return** res
 - 24: **function** $\text{Decode}_{\text{DCF}}(\llbracket f(x) \rrbracket_1, \dots, \llbracket f(x) \rrbracket_p)$ **return** $\sum_{i=1}^p \llbracket f(x) \rrbracket_i$
-

include the detailed algorithms. Algorithm 3 details the algorithms for this DCF scheme adapted from [10].

Similarly to [10], we can apply this technique either on the PRG-based schemes of [4] or on the trivial FSS schemes (i.e., sharing a truth table). The PRG-based DCF has $O(\sqrt[4]{N} \cdot \sqrt{q^{p^m}} \cdot \binom{p-1}{m} \cdot (\lambda + \log q))$ key size, while the information-theoretic DCF has $O(\sqrt{N} \cdot \binom{p-1}{m} \cdot \log q)$ key size.

Since our adapted DCF only adds a sub-DCF (assumed to be secure) compared to the initial DPF, the security proofs can be easily adapted from [10].

References

1. Barczewski, A., Mawass, A., Ramon, J.: Differentially Private Empirical Cumulative Distribution Functions (Feb 2025). <https://doi.org/10.48550/arXiv.2502.06651>, arXiv:2502.06651 [cs]
2. Boneh, D., Boyle, E., Corrigan-Gibbs, H., Gilboa, N., Ishai, Y.: Lightweight Techniques for Private Heavy Hitters. In: 2021 IEEE Symposium on Security and Privacy (SP). pp. 762–776 (May 2021). <https://doi.org/10.1109/SP40001.2021.00048>
3. Boyle, E., Couteau, G., Gilboa, N., Ishai, Y.: Function Secret Sharing and Homomorphic Secret Sharing (2022)
4. Boyle, E., Gilboa, N., Ishai, Y.: Function Secret Sharing. In: Advances in Cryptology - EUROCRYPT 2015. pp. 337–367. Lecture Notes in Computer Science (2015). https://doi.org/10.1007/978-3-662-46803-6_12
5. Boyle, E., Gilboa, N., Ishai, Y.: Function Secret Sharing: Improvements and Extensions. In: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security. pp. 1292–1303 (Oct 2016). <https://doi.org/10.1145/2976749.2978429>
6. Boyle, E., Gilboa, N., Ishai, Y.: Secure Computation with Preprocessing via Function Secret Sharing. In: Theory of Cryptography. pp. 341–371 (2019). https://doi.org/10.1007/978-3-030-36030-6_14
7. Boyle, E., Gilboa, N., Ishai, Y., Kolobov, V.I.: Information-Theoretic Distributed Point Functions. In: DROPS-IDN/v2/document/10.4230/LIPIcs.ITC.2022.17 (2022). <https://doi.org/10.4230/LIPIcs.ITC.2022.17>
8. Boyle, E., Gilboa, N., Ishai, Y., Kolobov, V.I.: Programmable Distributed Point Functions. In: Advances in Cryptology – CRYPTO 2022 (2022)
9. Bunn, P., Katz, J., Kushilevitz, E., Ostrovsky, R.: Efficient 3-Party Distributed ORAM. In: Security and Cryptography for Networks. vol. 12238 (2020). https://doi.org/10.1007/978-3-030-57990-6_11
10. Bunn, P., Kushilevitz, E., Ostrovsky, R.: CNF-FSS and Its Applications. In: Public-Key Cryptography – PKC 2022. vol. 13177, pp. 283–314 (2022). https://doi.org/10.1007/978-3-030-97121-2_11
11. de Castro, L., Polychroniadou, A.: Lightweight, Maliciously Secure Verifiable Function Secret Sharing. In: Advances in Cryptology – EUROCRYPT 2022. pp. 150–179 (2022). https://doi.org/10.1007/978-3-031-06944-4_6
12. Corrigan-Gibbs, H., Boneh, D., Mazières, D.: Riposte: An Anonymous Messaging System Handling Millions of Users. In: 2015 IEEE Symposium on Security and Privacy. pp. 321–338 (May 2015). <https://doi.org/10.1109/SP.2015.27>

13. Cramer, R., Damgård, I., Ishai, Y.: Share conversion, pseudorandom secret-sharing and applications to secure distributed computing. In: *Theory of Cryptography*. pp. 342–362 (2005)
14. Damie, M., Hahn, F., Peter, A., Ramon, J.: Eliminating Exponential Key Growth in PRG-Based Distributed Point Functions (Sep 2025). <https://doi.org/10.48550/arXiv.2509.22022>, arXiv:2509.22022 [cs]
15. Doerner, J., Shelat, A.: Scaling ORAM for Secure Computation. In: *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. pp. 523–535. CCS '17 (Oct 2017). <https://doi.org/10.1145/3133956.3133967>
16. Evans, D., Kolesnikov, V., Rosulek, M.: A Pragmatic Introduction to Secure Multi-Party Computation. *Foundations and Trends® in Privacy and Security* **2**(2-3), 70–246 (Dec 2018). <https://doi.org/10.1561/33000000019>
17. Gilboa, N., Ishai, Y.: Distributed Point Functions and Their Applications. In: *EUROCRYPT 2014*. pp. 640–658 (2014). https://doi.org/10.1007/978-3-642-55220-5_35
18. Goel, A., Wang, M., Wang, Z.: Multiparty Distributed Point Functions (2025)
19. Jawalkar, N., Gupta, K., Basu, A., Chandran, N., Gupta, D., Sharma, R.: Orca: FSS-based Secure Training and Inference with GPUs. In: *2024 IEEE Symposium on Security and Privacy (SP)* (2024), publication info: Published elsewhere. Minor revision. IEEE S&P 2024
20. Krips, T., Pullonen-Raudvere, P.: Multi-Party Distributed Point Functions with Polylogarithmic Key Size from Invariants of Matrices (2025)
21. Kruglik, S., Dau, S.H., Kiah, H.M., Wang, H., Zhang, L.F.: Verifiable Information-Theoretic Function Secret Sharing (2024), publication info: Preprint.
22. Kumar, C., Patranabis, S., Mukhopadhyay, D.: Compact Key Function Secret Sharing with Non-linear Decoder. *IACR Communications in Cryptology* **1**(2) (Jul 2024). <https://doi.org/10.62056/a3c3c3w9p>, number: 2
23. Li, J., Ke, P., Zhang, L.F.: Efficient Information-Theoretic Distributed Point Function with General Output Groups (2023), report Number: 625
24. Mouris, D., Sarkar, P., Tsoutsos, N.G.: PLASMA: Private, Lightweight Aggregated Statistics against Malicious Adversaries. *Proceedings on Privacy Enhancing Technologies* (2024)
25. Newman, Z., Servan-Schreiber, S., Devadas, S.: Spectrum: High-bandwidth Anonymous Broadcast. In: *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*. pp. 229–248 (Apr 2022)
26. Ryffel, T., Tholoniati, P., Pointcheval, D., Bach, F.: AriaNN: Low-Interaction Privacy-Preserving Deep Learning via Function Secret Sharing. *Proceedings on Privacy Enhancing Technologies* **1**, 291–316 (2022)
27. Servan-Schreiber, S., Beyzerov, S., Yablon, E., Park, H.: Private Access Control for Function Secret Sharing. In: *2023 IEEE Symposium on Security and Privacy (SP)*. pp. 809–828 (May 2023). <https://doi.org/10.1109/SP46215.2023.10179295>
28. Shamir, A.: How to share a secret. *Communications of the ACM* **22**(11), 612–613 (Nov 1979). <https://doi.org/10.1145/359168.359176>
29. Wagh, S.: Pika: Secure Computation using Function Secret Sharing over Rings. *Proceedings on Privacy Enhancing Technologies* p. 27 (2022)
30. Zyskind, G., Yanai, A., Pentland, A.S.: High-Throughput Three-Party DPFs with Applications to ORAM and Digital Currencies. In: *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security* (2024)